CYBERSECURITY PORTFOLIO PROJECT

API Security Testing Lab

Complete Presentation & Interview Guide

OWASP API Top 10 | FastAPI | Python | JWT Security | Rate Limiting

5 Vulnerabilities | 2 Modes | 30 Interview Q&As | Full Demo Script

Everything you need to present this project confidently to recruiters, lecturers, or engineers.

Table of Contents

60-Second Elevator Pitch

Say this when someone asks "what did you build?"

"I built a production-grade API security lab that demonstrates the OWASP API Security Top 10 vulnerabilities in a live, exploitable environment.

The system runs as a FastAPI application with a PostgreSQL database and Redis cache. It has two modes — vulnerable and secured — that you can switch in real time from the dashboard. In vulnerable mode, five real attacks work: you can enumerate every user's profile and steal their password hashes, escalate yourself to admin by sending `is_admin: true` in a request, forge JWT tokens with no signature, and brute force login indefinitely.

In secured mode, every one of those attacks is blocked — by ownership checks on the API layer, a strict Pydantic schema allowlist, RS256 JWT validation with expiry enforcement, and a Redis sliding-window rate limiter.

The codebase includes 30+ automated security tests that assert the vulnerabilities exist in one mode and are blocked in the other, a 7-stage CI/CD pipeline, Dockerised infrastructure, Kong API gateway, and a full set of offensive Python attack scripts."

System Architecture

What to draw on a whiteboard:

Key point to make: "The same codebase runs both modes. The `APP_MODE` environment variable is the only thing that changes. This is realistic — most real-world API vulnerabilities aren't separate codepaths; they're just the absence of a few lines of authorisation logic."

The 5 Vulnerabilities

1. BOLA — Broken Object Level Authorization (API1:2023)

CVSS: 8.1 HIGH

What it is:

The API returns data for any resource ID the client provides, without checking if the requesting user owns that resource.

What to say:

> *"This is the number one OWASP API vulnerability for two years running. The API uses sequential integer IDs — user 1, user 2, user 3. Once I'm authenticated as user 1, I can send `GET /api/users/2`, `GET /api/users/3` ... all the way to 15. The API has no ownership check, so it returns every user's profile including their password hash, admin flag, and internal UUID.*

> *In secured mode, the fix is a single conditional: `if current_user.id != requested_id and not current_user.is_admin: raise 403`. That one check makes 14 out of 15 requests return 403 Forbidden."*

What to show: BOLA card on the UI — run it in vulnerable mode, watch all 15 rows show green 200 with "LEAKED". Switch to secured mode — 14 rows flip to 403.

Real-world example: In 2019, Instagram exposed user private data through a BOLA vulnerability in their API. Researchers could access any user's profile by incrementing the user ID.

---

2. Mass Assignment (API3:2023)

CVSS: 7.5 HIGH

What it is:

The API blindly binds the entire request body to the ORM model, allowing attackers to set fields they shouldn't control — like `is_admin`.

What to say:

> *"In the vulnerable API, the update endpoint does this: it takes the raw JSON body as a Python dictionary and iterates over every key, doing `setattr(user, key, value)` for anything that matches a database column. So I register a normal user, then send a PUT request with `is_admin: true` . The API sets it. I'm now an admin.*

> *The secure fix is a Pydantic model with a strict allowlist — `UserUpdateSecured` only exposes `email` and `username` . Pydantic silently ignores every other field. The `is_admin` and `role` fields never reach the database."*

What to show: Mass assignment card — "Before" shows `is_admin: false` . After clicking Launch, "After" shows `is_admin: TRUE (escalated!)` . Switch to secured mode — "After" shows `false (blocked)` .

Real-world example: GitHub Enterprise had a mass assignment vulnerability in 2012 that allowed attackers to add SSH keys to any repository, giving full read/write access.

---

3. Excessive Data Exposure (API3:2023)

CVSS: 7.5 HIGH

What it is:

The API returns the full database object in responses, relying on the client to filter what it displays. Sensitive internal fields are sent over the wire even when not needed.

What to say:

> *"In the vulnerable mode, the users endpoint returns the ORM object directly — that means `password_hash` , `internal_id` , `login_count` , and `failed_login_count` all appear in every user response. The products endpoint exposes `internal_cost` — our cost price — to every customer. An attacker doesn't need to exploit anything; they just read the response.*

> *The fix is separate response schemas. `UserVulnerable` has 9 fields including the hash. `UserPublic` has 4 fields: id, username, email, is_active. FastAPI's `response_model` parameter enforces this at the serialization layer — even if the service layer accidentally returns extra fields, the schema strips them."*

What to show: Data Exposure card — in vulnerable mode, the table shows every field as EXPOSED in red. Switch to secured mode — everything shows as `hidden` in green.

Real-world example: In 2021, Peloton's API returned full user account objects including age, weight, workout history, and location to any unauthenticated request.

---

4. Broken Authentication — JWT Attacks (API2:2023)

CVSS: 9.8 CRITICAL

What it is:

The API uses a weak JWT configuration — predictable secret key, no expiry, and no algorithm validation — allowing attackers to forge tokens.

What to say:

> *"The vulnerable API signs JWTs with the string literal `'secret'` — that's crackable in milliseconds on a modern GPU. It also doesn't check the `exp` claim, so a token issued 10 years ago still works. Most critically, it doesn't enforce the signing algorithm, which enables the `none-algorithm` attack — I craft a JWT with `alg: none` in the header and no signature at all, and the API accepts it.*

> *The demo shows three attacks. The first two work in vulnerable mode: the none-alg token gets 200, and an expired token from an hour ago gets 200. The role-tamper attack fails in both modes because I used the wrong secret — that shows proper signature validation is working.*

> *The secure fix uses RS256 with a 2048-bit private key, enforces `exp` , `iss` , and `aud` claims, and calls `jwt.decode()` with strict validation. The algorithm is pinned to `RS256` — the library won't accept `none` as an algorithm."*

What to show: JWT card — in vulnerable mode, two of three attacks show ACCEPTED. In secured mode, all three show REJECTED (401).

Real-world example: In 2015, the `none` algorithm vulnerability was discovered across dozens of JWT libraries. It allowed complete authentication bypass — just set `alg: none` and omit the signature.

---

5. Unrestricted Resource Consumption — Rate Limiting (API4:2023)

CVSS: 7.5 HIGH

What it is:

The API places no limits on how fast or how many times a client can call it, enabling brute force, credential stuffing, and denial of service.

What to say:

> *"In vulnerable mode, I can fire 10,000 login attempts per minute against the API with wrong passwords. There's no 429, no lockout, no delay. A wordlist attack against `password1` through `password15` would succeed immediately.*

> *The secure fix is a Redis sliding-window rate limiter. The login endpoint is capped at 5 requests per minute per IP address. It uses a Lua script executed atomically in Redis to count requests in the current 60-second window. When you exceed the limit, you get a 429 with `Retry-After: 60` and `X-RateLimit-Remaining: 0` headers.*

> *I chose a sliding window over a fixed window because fixed windows have a boundary exploit — you can send 5 at 11:59 and 5 at 12:00 to get 10 in 2 seconds. A sliding window closes that gap."*

What to show: Rate limiting card — in vulnerable mode, all 10 attempts return 401 (no blocking). Switch to secured mode — attempts 6-10 return 429 Rate Limited with `Retry-After` header.

Real-world example: In 2020, a researcher demonstrated credential stuffing against a major streaming service's API, trying 300,000 username/password combinations without triggering any rate limiting.

Live Demo Script

Total demo time: 8–12 minutes

Opening (1 min)

1. Open `http://localhost:8001` — the dashboard loads in dark mode
2. Point to the header: "You can see we're in **VULNERABLE MODE** right now. This is a real FastAPI backend with 15 seeded users, 20 products, and 30 orders in a SQLite database."
3. Show the stats sidebar: users, products, orders, audit events

Attack Demo (6–8 min)

Click **"RUN ALL ATTACKS"** button and narrate as each result appears:

- **BOLA**: "15 out of 15 user profiles exposed. Notice the password hash — that's a real bcrypt hash. An attacker could take this offline and crack it."
- **Mass Assignment**: "Before: is_admin = false. After sending `is_admin: true` ... now it's true. We just gave ourselves admin rights through the API."
- **Data Exposure**: "Every sensitive field is marked EXPOSED. The API is returning the entire database row in the response."
- **JWT**: "Two out of three attacks worked. The none-algorithm token — no signature at all — was accepted."
- **Rate Limiting**: "10 out of 10 login attempts allowed. An attacker could try a million passwords without a single block."

Mode Switch (2–3 min)

1. Click **"SECURED"** toggle in the header — all cards reset
2. Click **"RUN ALL ATTACKS"** again
3. Narrate: "Same attacks. Different mode. Watch what changes."
4. Point to results: "BOLA — 14 out of 15 blocked. Mass assignment — is_admin stays false. Data exposure — every field hidden. JWT — all three rejected. Rate limiting — 429 after 5 attempts."
5. Say: "Zero code changes on the client side. The entire defence is in the API layer."

Closing (1 min)

"The same codebase runs both modes. The only difference is the `APP_MODE` environment variable. This is the real lesson — security isn't a separate product you bolt on. It's the absence of assumptions in your API layer."

Interview Q&A

Architecture & Design

Q: Why did you choose FastAPI over Flask or Django?

> *"FastAPI is async-native, which matters for I/O-heavy security workloads like Redis rate limiting and database queries. It also has native Pydantic integration which is central to the mass assignment fix — `response_model` enforcement happens at the serialization layer. Django REST Framework is more opinionated and heavier; Flask requires more setup for the same features."*

Q: Why SQLite in the demo instead of PostgreSQL?

> *"The production codebase uses async SQLAlchemy with PostgreSQL and runs via Docker Compose. The demo standalone uses SQLite purely for portability — no Docker required. The ORM abstraction means the security logic is identical either way."*

Q: What is the dual-mode architecture and why is it useful?

> *"A single codebase with `APP_MODE=vulnerable|secured` controlling three gating layers: which router is registered in `main.py`, which middleware is added to the stack, and how service functions branch. This is educationally valuable because it lets you do a side-by-side diff — the vulnerable and secured implementations are in the same file, making the exact fix visible."*

Q: How does the ObjectLevelAuthMiddleware work?

> *"It intercepts every request before it reaches the route handler. It uses regex patterns to detect resource-scoped paths like `/api/users/{id}`. It extracts the JWT, looks up the resource owner in the database, and compares it to the token's subject. If they don't match and the user isn't an admin, it short-circuits with a 403 and logs the violation to the audit table. Admins bypass the check entirely."*

Security Concepts

Q: What is BOLA and why is it API1:2023?

> *"BOLA — Broken Object Level Authorization — is when an API exposes a resource by its ID without verifying the requesting user is authorised to access it. It's been the top OWASP API risk for two consecutive years because it's so common: many developers implement authentication but forget authorisation. It's easy to test and easy to fix, yet it appears in production APIs constantly."*

Q: What's the difference between authentication and authorisation?

> *"Authentication answers 'who are you?' — it validates identity via JWT, OAuth token, API key. Authorisation answers 'what are you allowed to do?' — it checks that this specific user has permission to access this specific resource. BOLA is an authorisation failure, not an authentication failure. The attacker has a valid token; they're just using it to access resources they don't own."*

Q: Explain the JWT none-algorithm attack.

> *"JWT tokens have three parts: header, payload, signature. The header specifies the algorithm — usually HS256 or RS256. The `none` algorithm means no signature is required. A vulnerable JWT library honours the algorithm specified in the token's own header rather than the server's configured algorithm. So an attacker creates a token, sets `alg: none`, removes the signature, and sends it. The vulnerable library accepts it. The fix: always pin the algorithm server-side in `jwt.decode()` and never trust the algorithm from the token header."*

Q: What's the difference between HS256 and RS256?

> *"HS256 is symmetric — the same secret key signs and verifies tokens. Anyone with the key can both create and validate tokens. If the secret leaks, all tokens are compromised. RS256 is asymmetric — a private key signs tokens, a public key verifies them. The private key never leaves the server. Even if someone obtains the public key, they cannot forge new tokens."*

Q: What is a sliding window rate limiter and why is it better than a fixed window?

> *"A fixed window counter resets at fixed intervals — say every minute at :00. An attacker can send 5 requests at :59 and 5 more at :01, getting 10 requests in 2 seconds without exceeding the limit. A sliding window*

counts requests in the last N seconds from the current moment, not from a fixed boundary. This means the limit is always '5 in any 60-second window' regardless of where the window starts. I implement it with a Redis sorted set — timestamps as scores, removing any older than 60 seconds, then checking the count."

Q: Why is mass assignment dangerous beyond just setting is_admin?

"Beyond privilege escalation, an attacker can set fields like `password_hash` (overwrite someone's password without knowing it), `email` (account takeover via email reset), `login_count` (tamper audit records), `created_at` (falsify account age), or any custom field. In a financial API this could mean setting `account_balance`, `credit_limit`, or `interest_rate`. The fix is always explicit field allowlisting — only expose the fields users are meant to change."

Q: What is excessive data exposure and how is it different from BOLA?

"BOLA is about which resources you can access — user 1 accessing user 2's data. Excessive data exposure is about what fields are returned within a resource you're authorised to access. Even if you're only accessing your own profile, the API shouldn't return your `password_hash`, `internal_id`, or `failed_login_count`. The fix is response schemas — a dedicated output model that whitelists only the fields the client needs."

Implementation Details

Q: How does the rate limiter use Redis atomically?

"I use a Lua script executed with `redis.execute_script()`. Lua scripts in Redis are atomic — no other Redis command runs between script steps. The script takes the key, window size, and limit as arguments. It removes timestamps older than `now - window`, checks the count, conditionally appends the current timestamp, and returns the count. Doing this in a single atomic operation prevents race conditions where two concurrent requests both think they're under the limit."

Q: How do you test that the vulnerable mode is actually vulnerable?

"The pytest security tests use `@pytest.mark.vulnerable_mode` and `@pytest.mark.secured_mode` markers. The BOLA test in vulnerable mode asserts `status_code == 200` when accessing another user's resource. The same test in secured mode asserts `status_code == 403`. The mass assignment test sends `is_admin: true` and asserts the field changed in vulnerable mode and didn't change in secured mode. The CI pipeline runs both sets and fails if the secured mode has vulnerable behaviour or if the vulnerable mode doesn't exhibit the expected vulnerability."

Q: What are the 7 stages of your CI/CD pipeline?

"Stage 1: lint — ruff (formatting/style), mypy (type checking), bandit (SAST for Python). Stage 2: unit tests — no infrastructure, pure logic tests for JWT, password hashing, RBAC. Stage 3: integration tests — full Docker stack, tests against real endpoints. Stage 4 and 5 run in parallel — security tests (pytest against both modes) and OWASP ZAP active scan. Stage 6: Docker build and push to GHCR. Stage 7: deploy check — compose up + smoke test the health endpoint. The pipeline fails if secured mode has any vulnerable behaviour."

Q: How does Kong sit in front of the FastAPI app?

"Kong runs in DB-less declarative mode — no separate Postgres for Kong config, everything is in a YAML file. It proxies requests from port 8080 to the FastAPI app on port 8000. Kong handles the first layer of rate limiting — 5 req/min on `/auth/login`, 100/min on `/api/*`, 10/min on `/admin/*` using a Redis policy. It also adds CORS headers, injects a `Request-ID` header for tracing, limits request body size to 1MB, and has a Prometheus metrics endpoint."

Conceptual Questions

Q: If you were deploying this to production, what would you change?

> *"Several things: replace HS256 with RS256 even in 'vulnerable' mode for demo purposes — I'd just loosen the validation rather than use a weak algorithm. Add refresh token rotation so access tokens expire in 15 minutes. Add request signing for webhooks. Move secrets to AWS Secrets Manager or Vault rather than environment variables. Add a Web Application Firewall in front of Kong. Add structured logging to a SIEM like Splunk or Elasticsearch. Implement distributed tracing with OpenTelemetry."*

Q: What's the OWASP API Security Top 10 and why does it matter?

> *"It's OWASP's list of the most critical security risks specific to APIs, separate from the traditional OWASP Top 10 for web applications. It matters because APIs have different attack surfaces — they expose business logic directly, they often return structured data that's easy to parse, and they're consumed by machines not humans so there's no visual 'something looks wrong' signal. The list was first published in 2019, updated in 2023. My lab covers API1 through API4: BOLA, Broken Authentication, Broken Object Property Level Authorization, and Unrestricted Resource Consumption."*

Q: What would a penetration tester find using this system?

> *"In vulnerable mode: full user enumeration with credential data via BOLA (15 user hashes, admin flags), privilege escalation via mass assignment, full sensitive data dump from any API response, complete authentication bypass via JWT none-alg or expired tokens, and an unprotected brute-force surface. In a real engagement, those hashes go into hashcat, the admin escalation gets used to access* `/admin/*` *routes, and the data exposure gives the attacker the full internal data model to plan further attacks."*

Technical Deep-Dive

Database Design Choices

Sequential IDs vs UUIDs:

- Users table uses `id SERIAL` (sequential integers) — intentional for BOLA demonstration
- Also has `internal_id UUID` — shown as exposed in vulnerable mode
- In secured mode, products return `uuid` not `id` — demonstrates the UUID fix
- **Key point:** Sequential IDs alone aren't the vulnerability. The missing ownership check is. Even with sequential IDs, a proper authorisation check blocks BOLA.

Why SQLite in demo, PostgreSQL in production:

- SQLite: zero dependencies, file-based, same SQLAlchemy API
- PostgreSQL: JSONB columns for `shipping_address` and `audit_log.details`, `gen_random_uuid()` function, better concurrent write performance
- The SQLAlchemy ORM abstracts the difference — security logic identical

JWT Implementation Detail

```
# Vulnerable — no exp, no iss/aud, accepts none-alg
jwt.encode({"sub": "1"}, "secret", "HS256")
jwt.decode(token, "secret", algorithms=["HS256"],
```

```
          options={"verify_exp": False, "verify_aud": False})

# Secured — RS256, full claims, strict validation
jwt.encode({
    "sub": "1", "exp": now + 30min,
    "iss": "api-security-lab", "aud": "api-security-lab-users",
    "iat": now, "jti": uuid4()
}, private_key, "RS256")
jwt.decode(token, public_key, algorithms=["RS256"],
           audience="api-security-lab-users",
           issuer="api-security-lab")
```

Rate Limiter Lua Script

```
-- Redis sliding window — atomic, race-condition-free
local key = KEYS[1]
local now = tonumber(ARGV[1])
local window = tonumber(ARGV[2])
local limit = tonumber(ARGV[3])

-- Remove timestamps outside the window
redis.call('ZREMRANGEBYSCORE', key, 0, now - window)
local count = redis.call('ZCARD', key)

if count &lt; limit then
    redis.call('ZADD', key, now, now)
    redis.call('EXPIRE', key, window)
    return {1, limit - count - 1}  -- allowed, remaining
else
    return {0, 0}  -- blocked
end
```

Middleware Stack Order

```
Request in:
  1. AuditMiddleware     — log request (both modes)
  2. RateLimitMiddleware — check Redis window (secured only)
  3. ObjectLevelAuthMiddleware — check ownership (secured only)
  4. Route handler       — business logic

Response out:
  4 → 3 → 2 → 1 → client
```

Order matters: audit logs before rate limiting so blocked requests are still logged. Rate limiting before auth middleware to save DB lookups for blocked clients.

Design Decisions to Highlight

Decision	Why
Single codebase, env var gating	Shows the exact delta between vulnerable and secure — educational and demonstrates you understand what the fix IS
Both routers in same file	Direct side-by-side comparison — interview gold
Lua script for rate limiting	Demonstrates understanding of Redis atomicity and race conditions
RS256 over HS256	Shows awareness of symmetric vs asymmetric JWT tradeoffs
Pydantic response_model	Proves the fix is structural, not just removing fields manually
Sliding vs fixed window	Shows you understand the boundary exploit in fixed windows
ObjectLevelAuth as middleware	Centralised enforcement — can't be forgotten in a new route
AuditLog in both modes	Shows security logging is a baseline, not a premium feature

---

Quick Reference Cheat Sheet

```
VULNERABILITY          OWASP     CVSS   VULNERABLE MODE          SECURED MODE
──────────────────────  ────────  ──────  ──────────────────────  ──────────────────────────────
BOLA / IDOR            API1:23   8.1 H   No ownership check       id == current_user.id check
Mass Assignment        API3:23   7.5 H   setattr(user, any_key)   UserUpdateSecured schema
Excessive Data Expo.   API3:23   7.5 H   Return full ORM object   response_model allowlist
Broken Auth (JWT)      API2:23   9.8 C   HS256+"secret", no exp   RS256+exp+iss+aud enforced
Rate Limiting          API4:23   7.5 H   No limits                Redis sliding window 5/min

COMMANDS
──────────────────────────────────────────────────────────────────────────────
Start vulnerable:  APP_MODE=vulnerable uvicorn demo.demo_app:app --port 8001
Start secured:     APP_MODE=secured    uvicorn demo.demo_app:app --port 8001
Open UI:           http://localhost:8001/
Open API docs:     http://localhost:8001/docs
Switch mode (API): POST http://localhost:8001/demo/switch/secured
Run tests:         pytest tests/security/ -v
Run attacks:       python demo/run_attacks.py --url http://localhost:8001

KEY PEOPLE TO REFERENCE
──────────────────────────────────────────────────────────────────────────────
OWASP API Top 10:  Published by OWASP Foundation, last updated 2023
JWT none-alg:      Discovered by Tim McLean (2015), disclosed on auth0.com
Redis rate limit:  Described in "Redis in Action" by Josiah Carlson
BOLA research:     Documented extensively by API security researcher Alissa Knight
```

Built as a final-year project / portfolio piece demonstrating applied cybersecurity engineering.

Stack: FastAPI · SQLAlchemy · PostgreSQL · Redis · Kong · Docker · GitHub Actions · pytest